Session-open trading indicators for Freqtrade crypto bots

Ten session-aware strategies — ORB, VWAP, kill zones, CME gaps, and more — can be implemented in Freqtrade using vectorized pandas operations keyed to UTC timestamps on the candle `date` column. Every technique below maps the rhythms of traditional-finance session openings onto 24/7 crypto markets, exploiting the fact that institutional order flow still clusters around equity-market hours. The code targets Bybit futures with `can_short = True` on 5-minute candles, and all time-based logic derives from `dataframe['date'].dt.hour` rather than `datetime.utcnow()` — a critical Freqtrade pattern that keeps strategies valid in both backtesting and live trading. GitHub

---

1. Opening Range Breakout captures session momentum

The Opening Range Breakout (ORB) records the high and low during the first N minutes after a session opens, then trades the directional break of that range. QuantConnect In crypto, there is no opening bell — you define artificial session opens aligned with major financial centers.

Session open times and typical ORB windows:

Session	Open (UTC)	Common ORB durations
Asian	00:00	15m, 30m
London	07:00	15m, 30m
New York	13:30	5m, 15m, 30m

A **30-minute ORB** balances noise rejection with signal frequency and is the most common choice for crypto. Entry triggers when price closes above ORB high (long) or below ORB low

(short) after the window expires. Edgeful Stop loss sits at the opposite boundary; take-profit targets 1–2× the range width.

```python
```

```python
def populate_indicators(self, dataframe: DataFrame, metadata: dict) -> DataFram
    dataframe['hour'] = dataframe['date'].dt.hour
    dataframe['minute'] = dataframe['date'].dt.minute
    dataframe['mins'] = dataframe['hour'] * 60 + dataframe['minute']
    dataframe['session_date'] = dataframe['date'].dt.date

    orb_window = 30  # minutes
    sessions = {
        'ny':     {'open_mins': 13 * 60 + 30},
        'london': {'open_mins': 7 * 60},
        'asian':  {'open_mins': 0},
    }

    for name, cfg in sessions.items():
        om = cfg['open_mins']
        # Flag candles inside the ORB window
        dataframe[f'{name}_orb_window'] = (
            (dataframe['mins'] >= om) & (dataframe['mins'] < om + orb_window)
        ).astype(int)

        mask = dataframe[f'{name}_orb_window'] == 1
        dataframe.loc[mask, f'{name}_orb_high'] = (
            dataframe.loc[mask].groupby('session_date')['high'].transform('max'
        )
        dataframe.loc[mask, f'{name}_orb_low'] = (
            dataframe.loc[mask].groupby('session_date')['low'].transform('min')
        )
        # Forward-fill so post-ORB candles have the levels
        dataframe[f'{name}_orb_high'] = dataframe[f'{name}_orb_high'].ffill()
        dataframe[f'{name}_orb_low'] = dataframe[f'{name}_orb_low'].ffill()

        # Post-ORB trading window (~8 hours)
        dataframe[f'{name}_post_orb'] = (
```

```
            (dataframe['mins'] >= om + orb_window) &
            (dataframe['mins'] < om + 480)
        ).astype(int)

    return dataframe
```

Entry signals follow the standard breakout pattern — close above `orb_high` during `post_orb` triggers `enter_long = 1`, close below `orb_low` triggers `enter_short = 1`. Freqtrade The **Asian session** tends to build the range (lowest volatility), Acy while **London and NY** produce directional moves, Medium Aron Groups making ORB most profitable on those sessions.

---

2. Session VWAP resets at each open as dynamic support/resistance

VWAP — cumulative `Σ(typical_price × volume) / Σ(volume)` — is the institutional benchmark for "fair value." Medium A **session-resetting VWAP** anchors to each session open, providing a fresh mean-reversion line three times daily. Price above VWAP signals bullish bias; below signals bearish. Forex Tester Standard-deviation bands at **±1σ and ±2σ** mark overbought/oversold extremes.

The `pandas_ta` library offers a VWAP with an `anchor` parameter, GitHub but it only supports standard pandas offset aliases (`"D"`, `"W"`) and **cannot natively handle custom boundaries** like 07:00 or 13:30 UTC. Lightrun Freqtrade's built-in `qtpylib.rolling_vwap()` is windowed, not session-resetting. Custom implementation is required:

```python
```

```python
def assign_session_id(dates):
    """Create unique integer ID that increments at each session boundary."""
    ids, prev, counter = [], None, 0
    for dt in dates:
        mins = dt.hour * 60 + dt.minute
        if mins >= 810:    sess = 'ny'
        elif mins >= 420:  sess = 'london'
        else:              sess = 'asian'
        if sess != prev:
            counter += 1
            prev = sess
        ids.append(counter)
    return ids

def session_vwap(dataframe):
    df = dataframe.copy()
    df['tp'] = (df['high'] + df['low'] + df['close']) / 3.0
    df['sid'] = assign_session_id(df['date'])
    df['tp_vol'] = df['tp'] * df['volume']
    df['cum_tpv'] = df.groupby('sid')['tp_vol'].cumsum()
    df['cum_vol'] = df.groupby('sid')['volume'].cumsum()
    df['session_vwap'] = df['cum_tpv'] / df['cum_vol']

    # Standard deviation bands
    df['sq_dev'] = df['volume'] * (df['tp'] - df['session_vwap']) ** 2
    df['cum_sq'] = df.groupby('sid')['sq_dev'].cumsum()
    df['vwap_std'] = np.sqrt(df['cum_sq'] / df['cum_vol'])
    df['vwap_upper_1'] = df['session_vwap'] + df['vwap_std']
    df['vwap_lower_1'] = df['session_vwap'] - df['vwap_std']
    df['vwap_upper_2'] = df['session_vwap'] + 2 * df['vwap_std']
    df['vwap_lower_2'] = df['session_vwap'] - 2 * df['vwap_std']
    return df
```

Trading logic: A cross above session VWAP with RVOL > 1.5 is a high-conviction long entry. Mean-reversion entries trigger when price touches the ±2σ band and reverses. Note that VWAP **becomes sluggish late in a session** as cumulative data grows — the first and last 30 minutes around session opens produce the most actionable signals. CMC Markets

---

3. Volume profile, delta, and relative volume at open

Volume Profile distributes traded volume across price bins to identify the **Point of Control (POC)** — the price with the most volume, acting as a magnet — and the **Value Area** (70% of volume), whose high/low boundaries serve as support and resistance. GitHub Trading-strategies The `py-market-profile` library (`pip install MarketProfile`) calculates POC and value area from OHLCV DataFrames natively. GitHub PyPI For a lighter-weight approach, distribute each candle's volume uniformly across its high-low range into bins:

```python
def volume_profile(df, num_bins=30):
    bins = np.linspace(df['low'].min(), df['high'].max(), num_bins + 1)
    centers = (bins[:-1] + bins[1:]) / 2
    vols = np.zeros(num_bins)
    for _, r in df.iterrows():
        lo, hi = max(0, np.searchsorted(bins, r['low']) - 1), min(num_bins - 1,
        if hi >= lo:
            vols[lo:hi+1] += r['volume'] / (hi - lo + 1)
    poc_idx = np.argmax(vols)
    return {'poc': centers[poc_idx], 'bins': centers, 'volumes': vols}
```

Volume Delta approximates buy/sell pressure from OHLCV with the formula `delta ≈ volume × (close - open) / (high - low)`. Positive delta indicates aggressive buying; negative signals selling pressure. Medium Cumulative delta within a session reveals whether buyers or sellers are dominating. True taker-side data requires Bybit's `/v5/market/recent-trade` endpoint, which Freqtrade's standard pipeline does not expose.

Relative Volume (RVOL) compares current volume to the average for the **same time of day** across N prior sessions — simple `current_vol / SMA(vol, 20)` is misleading because volume varies dramatically by session. The correct approach: TradingView

```python
def rvol_at_time(dataframe, lookback=20):
    dataframe['time_key'] = dataframe['date'].dt.strftime('%H:%M')
    dataframe['rvol'] = dataframe.groupby('time_key')['volume'].transform(
        lambda x: x / x.rolling(lookback, min_periods=3).mean()
    )
    return dataframe
```

RVOL > 1.5 at a session open indicates unusual institutional activity and serves as a powerful confirmation filter for any breakout signal. **RVOL > 2.0** often precedes sustained momentum moves. Plus500

---

4. Initial Balance sets the day-type framework

The **Initial Balance (IB)** — from J. Peter Steidlmayer's Market Profile — is the high-low range of the first 60 minutes after a session opens. FTMO IB width predicts day type: a **narrow IB** (below the 20-session rolling average × 0.7) signals a likely trend day and favors breakout trades, while a **wide IB** signals range-bound conditions and favors mean-reversion. ATAS

Key IB-derived levels for targets and S/R:

- **IB Midpoint** — intraday pivot
- **1.5× IB extension** — first take-profit target
- **2.0× IB extension** — aggressive target on trend days

The **IB failure setup** offers an edge: if price breaks the IB boundary but fails to close outside it, there is a **70–75% probability** it will trade to the opposite boundary. Tradeproacademy Implementation mirrors the ORB code but with a 60-minute window. The `py-market-profile` library provides `mp_slice.initial_balance()` for direct calculation from OHLCV DataFrames. GitHub PyPI

```python
# IB extension levels for take-profit targeting
dataframe['ib_range'] = dataframe['ny_ib_high'] - dataframe['ny_ib_low']
dataframe['ib_ext_1_5_up'] = dataframe['ny_ib_high'] + 1.5 * dataframe['ib_rang
dataframe['ib_ext_2_0_up'] = dataframe['ny_ib_high'] + 2.0 * dataframe['ib_rang
dataframe['ib_ext_1_5_dn'] = dataframe['ny_ib_low'] - 1.5 * dataframe['ib_range
```

---

5. ICT Kill Zones define when to trade, not just what

Kill Zones — from Michael Huddleston's ICT methodology — are specific UTC windows when institutional order flow concentrates. **The timing is primary; the setup is secondary.** Ictkillzone A technically perfect entry outside a kill zone has significantly lower probability. Ictkillzone

Kill Zone	UTC (during US DST)	Role
Asian KZ	00:00–04:00	Accumulation — builds range, creates liquidity
London Open KZ	06:00–09:00	Manipulation — sweeps Asian range, sets daily direction
NY Open KZ	12:30–15:00	Distribution — highest probability, real move
London Close KZ	14:00–16:00	Profit-taking, reversals
Silver Bullet	14:00–15:00	Single highest-probability hour

The **Power of Three (PO3)** maps directly: Asian session = accumulation, London open = manipulation (Judas Swing sweeps one side of the Asian range), NY session = distribution (the real directional move). XS +2 The most powerful use of kill zones is as a **filter overlay** on other strategies:

```python
# Add kill zone filter to ANY existing entry logic
dataframe.loc[
    (your_existing_long_condition) &
    ((dataframe['in_london_kz'] == 1) | (dataframe['in_ny_kz'] == 1)),
    'enter_long'] = 1
```

Implementation flags each candle's kill zone membership using minute-of-day comparisons. **DST adjustment matters**: kill zones are defined in New York local time, so UTC boundaries shift by one hour between March and November. Ictkillzone Use a wider

window (e.g., 06:00–10:00 for London) to cover both DST states, or derive NY-local time with `zoneinfo.ZoneInfo('America/New_York')`.

---

6. CME gap fill strategies — effective now, expiring May 2026

CME Bitcoin futures gaps fill approximately 77% of the time, Phemex with ~74% filling within 5 trading days BitcoinWorld and ~85% of gaps under $500 filling within 1–2 weeks. Phemex The strategy is mean-reversion: a gap up (Sunday open > Friday close) creates short bias; a gap down creates long bias. Millionero Magazine CME's Friday close approximates at **21:00 UTC** and Sunday reopen at **22:00 UTC.** Phemex

⚠️ **Critical timing note:** CME announced on February 19, 2026 that it will launch **24/7 cryptocurrency futures trading starting May 29, 2026,** CME Group with only a 2-hour weekend maintenance window. Cointelegraph This will effectively eliminate the classic CME gap phenomenon after that date. Phemex

Since Freqtrade on Bybit doesn't access CME data directly, approximate CME close by recording the Bybit BTC/USDT price at 21:00 UTC Friday:

```python
```

```
# In populate_indicators()
dataframe['is_fri_close'] = (
    (dataframe['date'].dt.weekday == 4) & (dataframe['hour'] == 20)
).astype(int)
dataframe['cme_fri_close'] = np.where(
    dataframe['is_fri_close'] == 1, dataframe['close'], np.nan
)
dataframe['cme_fri_close'] = dataframe['cme_fri_close'].ffill()

dataframe['is_sun_open'] = (
    (dataframe['date'].dt.weekday == 6) & (dataframe['hour'] == 22)
).astype(int)
dataframe['cme_gap'] = np.where(
    dataframe['is_sun_open'] == 1,
    dataframe['open'] - dataframe['cme_fri_close'], np.nan
)
dataframe['cme_gap'] = dataframe['cme_gap'].ffill()
dataframe['cme_gap_pct'] = dataframe['cme_gap'] / dataframe['cme_fri_close'] *

# Gap fill detection
dataframe['gap_filled'] = (
    ((dataframe['cme_gap'] > 0) & (dataframe['low'] <= dataframe['cme_fri_close
    ((dataframe['cme_gap'] < 0) & (dataframe['high'] >= dataframe['cme_fri_clos
).astype(int)
```

Risk management: minimum gap threshold of **0.3–0.5%** filters noise. Hard stop at −4% or 1.5× the gap size. Time stop at 168 hours (1 week). Conservative leverage of 2–3× since gap trades are mean-reversion. **Avoid gap trades in strong trending markets** (EMA 50 well separated from EMA 200) — breakaway gaps in trends may not fill for months.

CoinMarketCap

7. London Breakout adapted for crypto

The London Breakout trades the breakout of the Asian session range (DailyForex) (00:00–07:00 UTC high/low) at the London open. The strategy filters for **narrow Asian ranges** (below 1.2× the 20-session average) indicating consolidation, then enters when a 5m candle closes beyond the range during the 07:00–10:00 UTC entry window.

Targets are set at **1.5× the Asian range width** beyond the breakout level. (DailyForex) Stop loss sits at the opposite side of the range (or the midpoint for tighter risk). A **time-based exit at 16:00 UTC** prevents overnight carry.

The false-breakout variant may outperform in crypto. Because crypto markets are prone to liquidity sweeps, London often *sweeps* one side of the Asian range, then reverses. (Ictkillzone +2) This aligns with the ICT Power of Three framework:

```python
# False breakout reversion: enter AGAINST the sweep
dataframe['swept_high_failed'] = (
    (dataframe['high'] > dataframe['asian_high']) &     # Wick above
    (dataframe['close'] < dataframe['asian_high'])      # Close back below
).astype(int)
# → Enter SHORT after failed high sweep

dataframe['swept_low_failed'] = (
    (dataframe['low'] < dataframe['asian_low']) &
    (dataframe['close'] > dataframe['asian_low'])
).astype(int)
# → Enter LONG after failed low sweep
```

Key filters for the standard breakout: **volume confirmation** (breakout candle volume > 1.2× the 20-period SMA), RSI not overbought/oversold, (NYC Servers) and Asian range between

0.2% and 1.5% of price to avoid dust trades and already-extended ranges. No published crypto-specific backtests exist for this strategy — forward testing is essential.

---

8. Libraries, tools, and Freqtrade implementation patterns

Python libraries for session-aware indicators:

Library	Key function	Use case
`talib`	`ATR`, `EMA`, `RSI`, `SMA`	Standard filters, trend context
`pandas-ta`	`vwap(anchor="D")`, `atr`, `bbands`	Daily VWAP (custom sessions need manual code) Libraries.io
`MarketProfile` (`pip install MarketProfile`)	`initial_balance()`, POC, value area	Market Profile / IB from OHLCV
`qtpylib` (Freqtrade built-in)	`rolling_vwap()`	Windowed VWAP (not session-resetting)

Critical Freqtrade patterns for session strategies:

- **Never use** `datetime.utcnow()` in `populate_*` methods — it breaks backtesting. DEV Community GitHub All time logic must derive from `dataframe['date'].dt.hour`. github
- **Vectorized only** in `populate_*` — no Python loops, no `iloc[-1]`. Use `.shift()` for previous-candle references freqtrade and `groupby().transform()` for session aggregations. Freqtrade

- **Forward-fill after session close, not during** — computing Asian high/low and forward-filling immediately creates lookahead bias if the session hasn't ended. Condition the ffill to only apply when `hour >= session_end_hour`.
- `startup_candle_count >= 300` for session strategies — ensures enough history for 20-session rolling averages on 5m candles. (Freqtrade)
- `process_only_new_candles = True` for performance (GitHub) — session calculations can be expensive.
- **Candle timing:** signals generate at candle close; trades open at the next candle's open (one-candle delay). (Freqtrade) Use `--timeframe-detail 1m` in backtesting for more realistic fill simulation. (Freqtrade)
- **Multi-timeframe data** via `informative_pairs()` + `merge_informative_pair()` — informative data is auto-shifted by one candle to prevent lookahead. (Freqtrade)

Combining everything into a unified SYGNIF strategy

The highest-edge approach stacks these indicators into a layered decision framework rather than using any single strategy in isolation:

1. **Kill zone filter** — only trade during London or NY kill zones (eliminates ~60% of low-probability setups)
2. **IB width classifier** — narrow IB → breakout mode, wide IB → mean-reversion mode
3. **ORB or London Breakout** for entry timing — trade the first clean break of the session range
4. **Session VWAP** as directional bias — long only above VWAP, short only below
5. **RVOL > 1.5** as conviction filter — confirms institutional participation
6. **Volume delta** for order-flow confirmation — positive delta for longs, negative for shorts

7. **CME gap bias** (until May 29, 2026) as a higher-timeframe overlay — trade in the direction of gap fill when an unfilled gap exists

This stacking approach ensures each signal has multiple layers of confirmation. The session-awareness creates a structural edge: you're trading when institutions are active, in the direction their flow supports, confirmed by volume. Set `can_short = True`, leverage at 2–3× maximum, and implement per-session position limits to prevent overexposure to correlated session signals.